

E1 · Industrialiser les données de l'entreprise

Alternance : dataplateforme et interfaces de collecte



MongoDB · ServiceNow ·
Elasticsearch
Extraction orchestrée chaque
jour par Airflow



ClickHouse · dbt
Typage, nettoyage, modèles et
marts

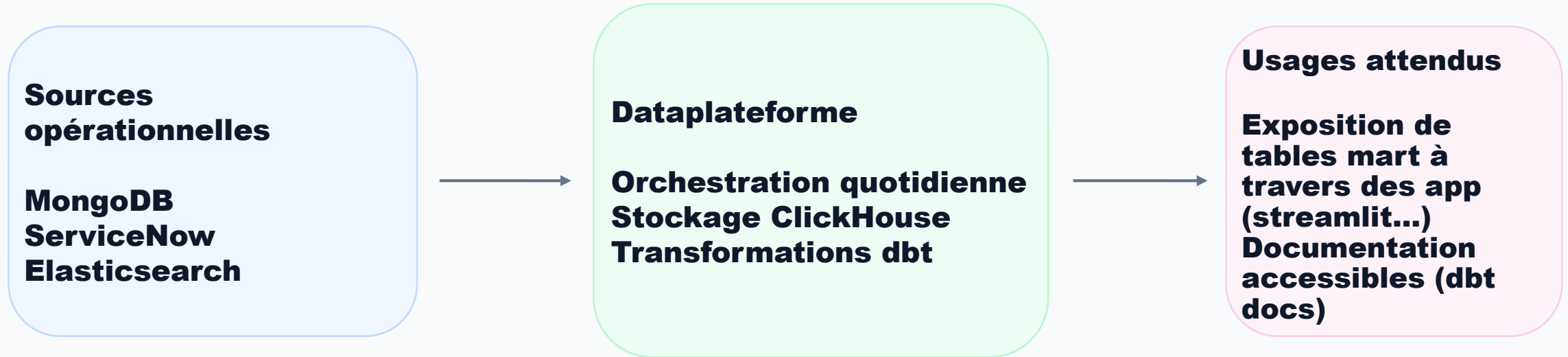


Documentation · accès métier

Contexte : transformer les données internes en usages métier

E1 · Bloc 1 · C1-C5

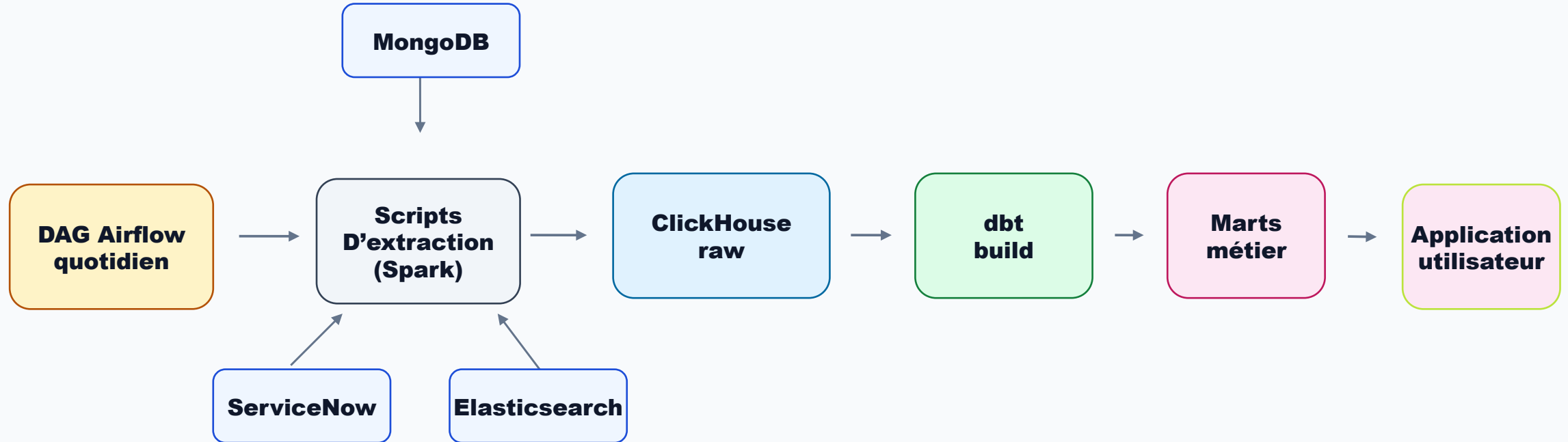
Point de départ : plusieurs sources opérationnelles, pas directement exploitables par les équipes.



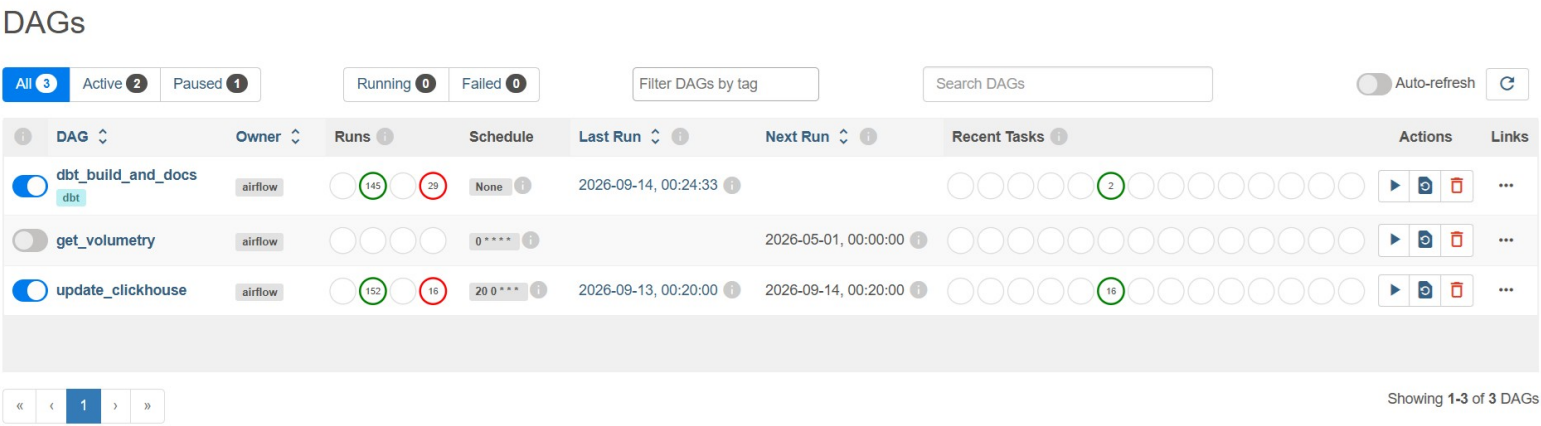
Architecture réalisée : du run quotidien au mart consommable

E1 · Bloc 1 · C1-C5

Le trajet de la donnée se lit de gauche à droite

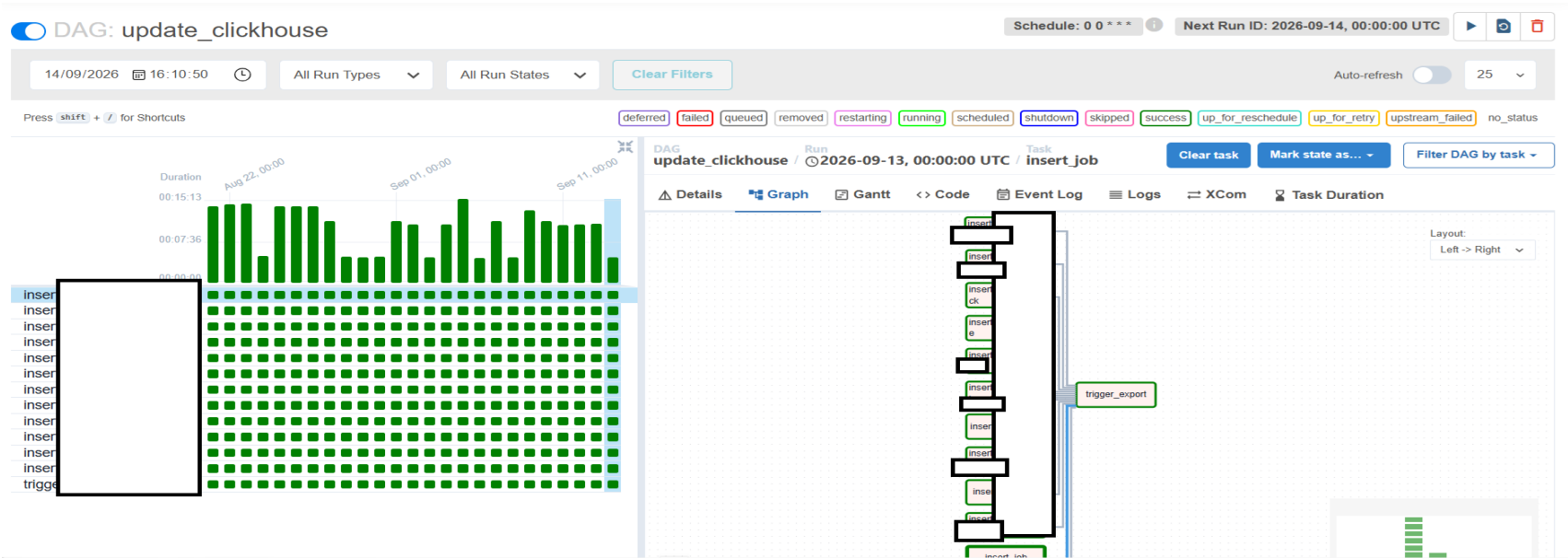


C1 - Architecture réalisée : du run quotidien au mart consommable



Airflow :

- Dag d'ingestion des données quotidien
- Dag de run dbt quotidien avec données du jour



C1 - Collecte : trois sources, trois contraintes d'extraction

E1 · Bloc 1 · C1-C5

La preuve n'est pas la diversité des noms de sources ; c'est le mécanisme de récupération réellement programmé.

MongoDB

Challenges:

- *Documentation pauvre
- *Champs anormaux (JSON...)

ServiceNow

Challenges:

- *Données représentées en XML
- *Valeurs initialement textuelles
- *Erreur de type avant conversion
- *Documentation pauvre ou inexistante

Elasticsearch*

Challenges:

- *Requêtes d'extraction
- *Périmètre, filtres, pagination
- *Gestion des erreurs/volumes

```
def fetch_snow_table(
    spark_session: SparkSession, instance: str, session: requests.Session, config: TableConfig, limit: int = 50000
) -> DataFrame:
    """
    Fetch the Snow Database and returns a DataFrame containing all columns provided from the given table.

    Args:
        instance (str): instance of the Snow client.
        session (requests.Session): Request session for the Snow client.
        config (TableConfig): Configuration object of the table.
        limit (int, optional): Maximum amount of rows to query from Snow.

    Returns:
        DataFrame: Spark DataFrame with the data from the given table.
    """
    try:
        # Adds the columns and limit to the params builder
        params_builder = ParamsBuilder()

        if config.columns is not None:
            params_builder.fields = list(config.columns)

        params_builder.limit = limit
        params_builder.display_value = True
```

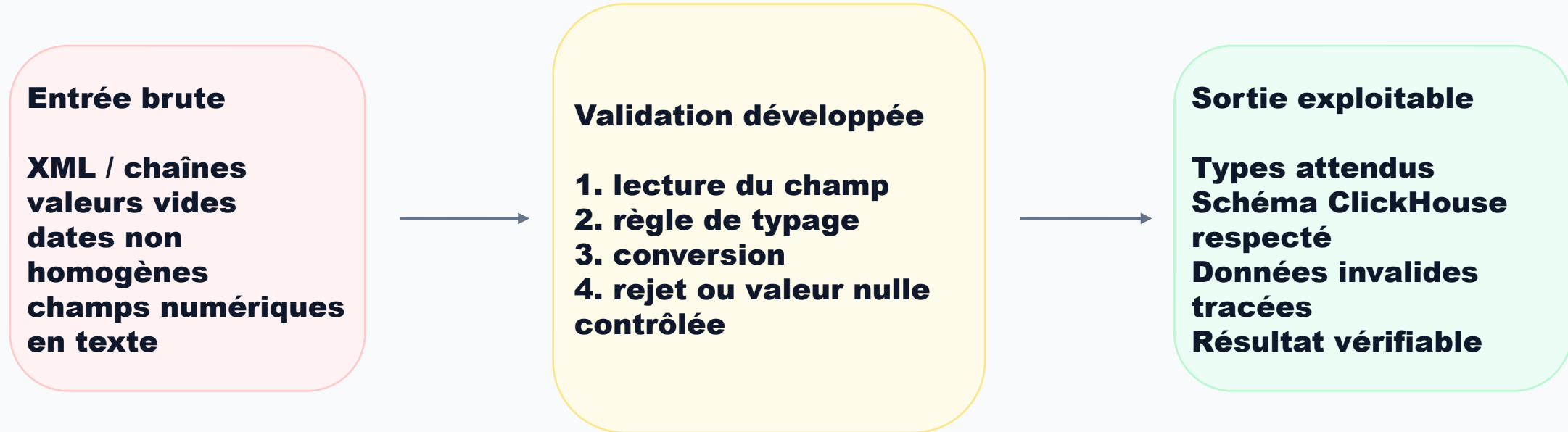
```
def fetch_mongo_table(
    spark_session: SparkSession, mongo_uri: str, config: TableConfig) -> DataFrame:
    """
    Fetch the Mongo Database and returns a DataFrame containing all columns provided from the given table.
    Args:
        config (TableConfig): Configuration object of the table.

    Returns:
        DataFrame: Spark DataFrame with the data from the given table.
    """
    try:
        spark_schema = build_spark_schema(config.columns)

        # Creates the reader
        reader = (
            spark_session.read.format("mongodb")
            .schema(spark_schema)
            .option("database", config.database)
            .option("collection", config.source_table)
            .option("connection.uri", mongo_uri)
        )
```

C2-C3 - Nettoyer et normaliser avant de stocker

E1 · Bloc 1 · C1-C5



```

{{ config(
  materialized = 'incremental',
  incremental_strategy='delete+insert',
  unique_key = [
    'id'
  ],
  on_schema_change = 'append_new_columns',
  enabled=true
) }}

WITH joined AS (
  SELECT
    {{ ref('date_created') }} AS date_created,
    {{ ref('name') }} AS name AS customer,
    {{ ref('customer') }} AS customer,
    {{ ref('id') }} AS id
  FROM {{ ref('mongo_raw') }} AS mongo_raw
  LEFT JOIN {{ ref('mongo_raw') }} AS mongo_raw_2
    ON mongo_raw.id = mongo_raw_2.id
  WHERE mongo_raw.id IS NOT NULL AND mongo_raw_2.id IS NOT NULL
)
SELECT
  ['Lundi', 'Mardi', 'Mercredi', 'Jeudi', 'Vendredi', 'Samedi', 'Dimanche'][toDayOfWeek(date_created)] AS day,
  toStartOfHour(date_created) AS date,
  toHour(date_created) AS hour,
  {{ ref('id') }} AS id,
  {{ ref('name') }} AS name,
  {{ ref('customer') }} AS customer,
  count(*) AS n_trigger
FROM joined
GROUP BY day, date, hour, id, name, customer

```

Table dbt (fichier .sql)

```

database: clickhouse
source_table: mongo_raw
dbt_table_name: mongo_raw
time_field: "date_create"
days_to_fetch: 7
ingest_daily: true
increment_strategy: delete+replace_per_day
columns:
  id:
    type: string
    nullable: false
  date_create:
    type: string
    nullable: true
  date_inserted:
    type: timestamp
    nullable: false
  name:
    type: string
    nullable: true
  customer:
    type: string
    nullable: true
  is_trigger:
    type: bool
    nullable: true

```

Configuration d'une table raw (extraction ; fichier .yaml)

```

def generate_create_raw_table_sql(
    config: dict,
    order_by: str = "_id"
) -> str:
    """
    Generates a ClickHouse CREATE TABLE statement for a raw Mongo ingestion table.
    """
    validate_raw_table_config(config.columns)

    known_columns = set(config.columns) | {"time_insert_to_clickhouse"}

    if order_by not in known_columns:
        raise ValueError(f"ORDER BY column {order_by!r} is not present in the table.")

    table_name = build_raw_table_name(config.database, config.dbt_table_name)

    column_lines = []

    for column_name, column_config in config.columns.items():
        ch_type = to_clickhouse_type(column_config)
        column_lines.append(f"    {clickhouse_escape_identifier(column_name)} {ch_type}")

    column_lines.append("    `time_insert_to_clickhouse` DateTime64(6)")

    columns_sql = ",\n".join(column_lines)
    order_by_sql = f"time_insert_to_clickhouse, {order_by}"

    return f"""
CREATE TABLE IF NOT EXISTS {clickhouse_escape_identifier(table_name)}
(
    {columns_sql}
)
ENGINE = MergeTree
ORDER BY ({order_by_sql})
TTL `time_insert_to_clickhouse` + INTERVAL 30 DAY
""".strip()

```

Scripte de génération des tables raw dans Clickhouse (python)

C4 - Stockage : ClickHouse, DBT

E1 · Bloc 1 · C1-C5

Choix ClickHouse :

Volumes de données élevés

Requêtes rapides

Concurrence

CLI efficace et intuitif

Facile à dockeriser

Autres DB testées et rejetées :

DuckDB (Pas de concurrence)

DELTA (Difficile à maintenir (.jar Spark), contraintes projet (plus difficile à dockeriser et à exploiter))

Conception et architecture :

Schémas raw reproduit des bases d'origine après data analyse.

Toutes les colonnes ne sont pas récupérées.

Schémas raw stockés dans fichiers .yaml de configuration

Les tables intermédiaires et mart sont modélisées selon les besoins finaux

Chaque table mart répond à un use case produit ou équipe

Schémas en étoile :

Les marts utilisent des tables de dimension

Schéma global :

Raw (1:1 origine) → Staging (transformation légère ;

renaming ; extraction donnée complexe (JSON) →

Intermédiaire (aggrégats, enrichissements, jointures) →

Mart (tables finales)



Search for models...

data_warehouse

macros

common_tables

compatibility

create_timestamp_table

timestamp_post_hook

timestamp_value

models

intermediate

marts

fact_

fact_

fact_

fact_

fact_

fact_

fact_

fact_

fact_

fact_

staging

snapshots

fact_

incremental

DetailsDescriptionColumnsDepends OnCode

Columns

COLUMN	TYPE	DESCRIPTION	CONSTRAINTS	DATA TESTS	MORE?
customer	String				>
	String				>
	String				>
	String				>
	String				>
	Nullable(Boolean)				
	Nullable(Integer32)				>
	Nullable(Boolean)				>
	String				>
	Nullable(Integer32)				

Documentation interactive générée par dbt (anonymisée)

Lineage Graph



Lignage des données (en vert les tables sources ou « raw ») anonymisé

Table des

Data last updated at : 14/09/2026 00:00

Run

Id		Version
6a1942ef24246e7b6e933527		1.5
68c7d34b8879cec063489dcc		1.3
692468c29619470cd6b1eec1		1.8
692721079c7c971de8be31e6		1.0
692722f260011fea42e88650		1.0
69d77962df1efdeaca3053cf		1.6
6a227fd192704ec8373ca714		1.3
6a43cf9157c54a98c32de662		1.3
6a70e72db41f07c07e9368e4		1.4
6a7351c8d1013cdf7eb98c8a		1.3

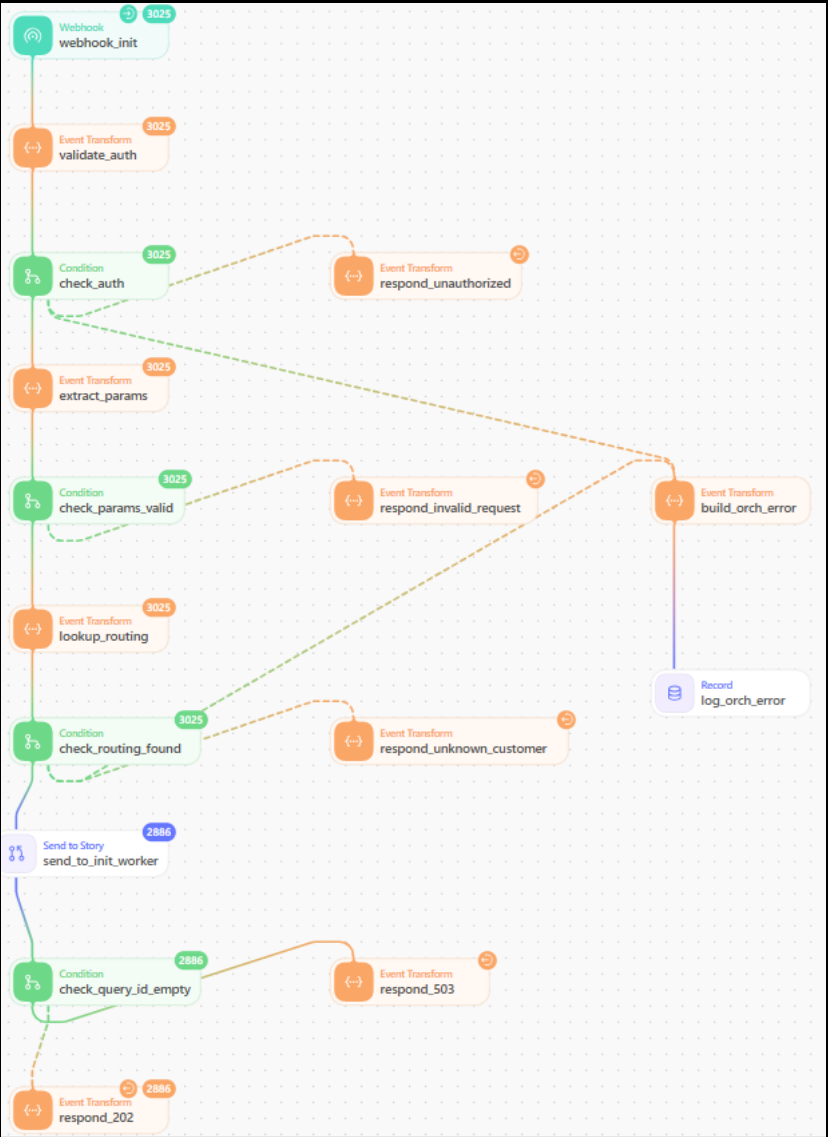
Note: interactive sorting is disabled above 150k rows.

Download as CSV

Preparing the Excel file...

Running `to_excel_bytes(...)`.

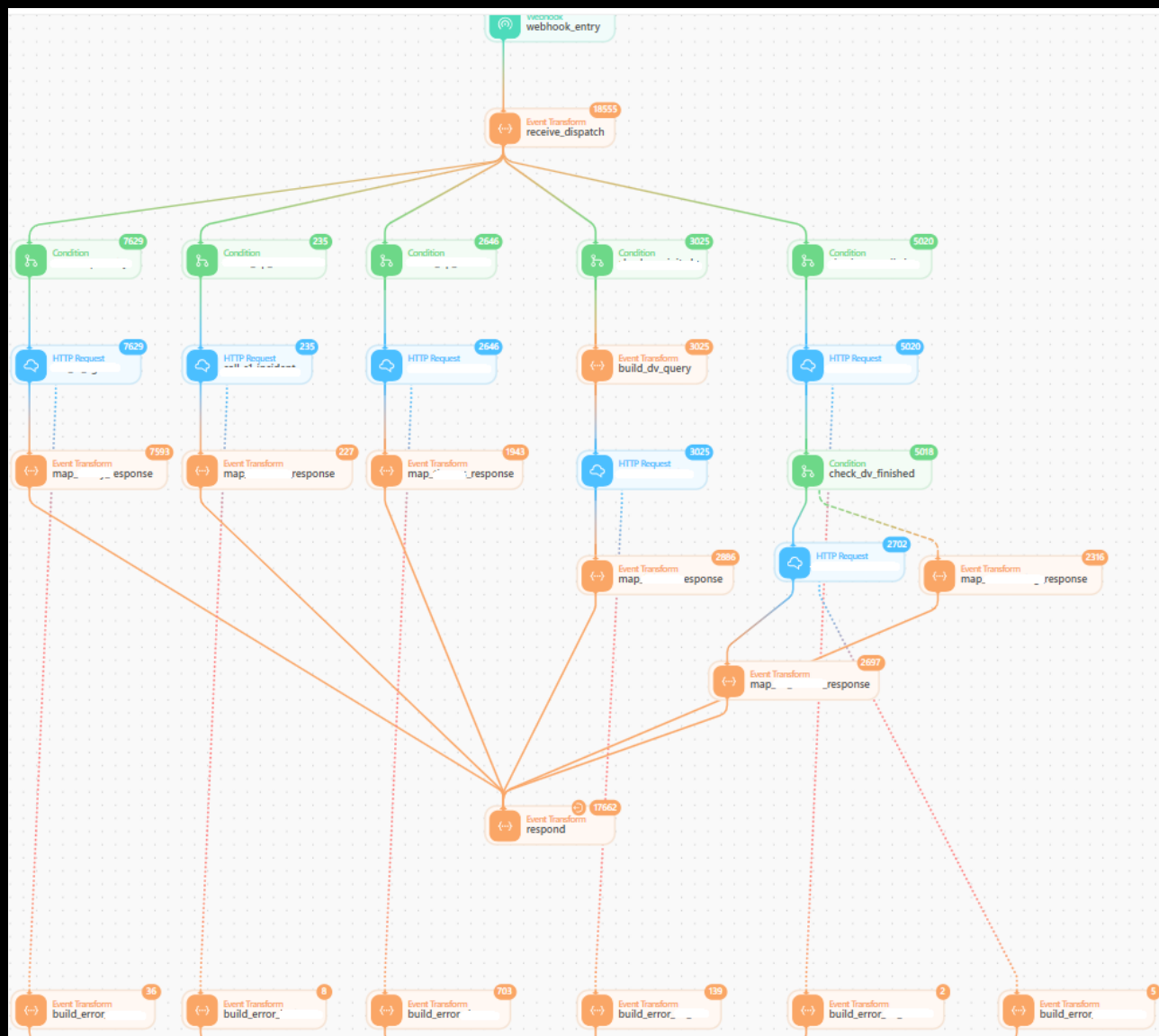
Application streamlit permettant l'obtention des tables mart. Exports disponibles en csv/excel pour dashboard. (Anonymisé)



Exemple de Story (= endpoint) Tines

C5 - Mise à disposition

E1 · Bloc 1 · C1-C5



Story Times « worker » interrogeant les endpoints API des consoles EDR après réception des paramètres de requêtes des stories « endpoint ». (Anonymisé)

Call To Aldventure Monster Dataset API 1.0.0 OAS 3.0

/api/v1/openapi.yaml

Read access to the normalized monster dataset and staff access to ingestion summaries.

Authorize

V

default

^

GET	/api/v1/monsters/	List normalized monsters	
GET	/api/v1/monsters/{monster_id}/	Retrieve a monster and its source provenance	
GET	/api/v1/ingestion-runs/{run_id}/summary/	Retrieve an ingestion summary (staff session required)	

API Rest – Call to Aldventure

Exploitation : maintenir la chaîne quand volume et schémas évoluent

E1 · Bloc 1 · C1-C5

Charge

Tables de quelques dizaines de milliers à plusieurs dizaines de millions de lignes.

```
SELECT count(*)
FROM [redacted]

Query id: fc8679e6-749b-4191-a488-3df7201d0c79

1. [redacted] -- 11.06 million

1 row in set. Elapsed: 0.012 sec.
```

Fenêtres dynamiques

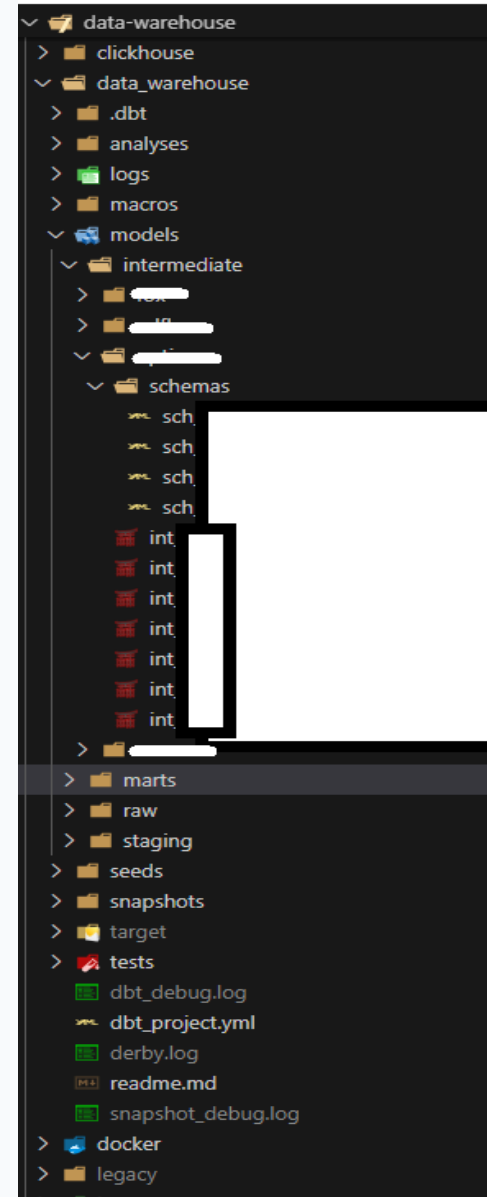
Plages de récupération quotidiennes ajustées selon la table et le volume.

Schéma évolutif

Ajout contrôlé de colonnes absentes des schémas ClickHouse.

Maintenance

Correction de bugs, demandes d'évolution, documentation de reprise.



Architecture du projet dbt

Bilan : couverture des compétences C1 à C5

E1 · Bloc 1 · C1-C5

Une réalisation principale data, complétée par une preuve d'interface programmable clairement séparée.

C1

Extraction automatisée

DAG + scripts + sources hétérogènes

C2

SQL d'extraction/transformation

Extraits SQL/dbt commentés

C3

Nettoyage et agrégation

ServiceNow typé + mart métier

C4

Base de données et import

ClickHouse + modèle + documentation

C5

Mise à disposition

Streamlit + webhooks Tines + API Rest